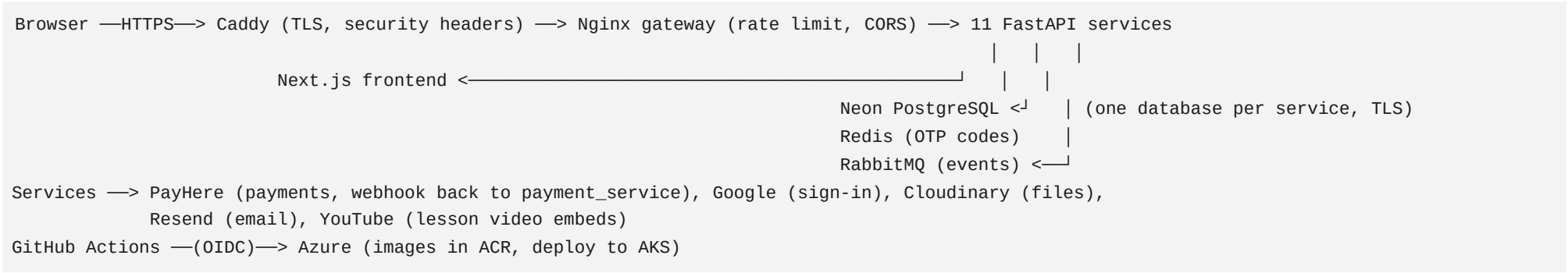
Threat Model

Guideline reference: §8.1 (OWASP A04, Insecure Design: "threat model documented in the SDS security section"). Written to be added to the SDS security section; the SDS itself is not kept in this repository.

- **Date:** 2026-10-28
- **Scope:** the EDURA platform as deployed on staging (`https://staging.eduralms.com`): web frontend, gateway, 11 backend services, data stores, third-party services and the delivery pipeline.
- **Method:** a data-flow description, then STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) applied to each trust boundary. Controls and gaps are taken from the code and configuration (see `owasp-checklist.md` for the evidence); nothing here was penetration-tested.

1. System and data flows



Actors: anonymous visitors, students, teachers and administrators, the payment provider's server, and the team's operators.

Assets

Asset	Why it matters
Account credentials and session tokens	Account takeover, privilege escalation
JWT signing key (`SECRET_KEY`, shared by all services)	Anyone holding it can issue a token for any user and role
Student personal data (name, phone, school, NIC for grades 12 to 13, email)	Privacy; the users include minors
Payment records, receipts and webhook integrity	Money and access to paid content
Exam questions, correct answers, submissions and rankings	Fairness and integrity of results
Paid course content (video ids, notes)	Revenue

Asset	Why it matters
Secrets in GitHub (database URLs, API keys, RabbitMQ password) and the cluster	Control of the whole system

2. Trust boundaries

#	Boundary	Crossed by
TB1	Internet to the edge (Caddy and the gateway)	All browser traffic, the PayHere webhook
TB2	Gateway to the services (inside the cluster)	Proxied API requests
TB3	Service to service and to RabbitMQ and Redis	Events and cached codes
TB4	Services to Neon PostgreSQL (public network, TLS)	All data access
TB5	Services to third parties	PayHere, Google, Cloudinary, Resend
TB6	Delivery pipeline to the cluster	Images, manifests, secrets

3. Threats, controls and residual risk

#	STRIDE	Threat	Existing controls	Residual risk / gap	Ref
T1	Spoofing	Guessing or stuffing passwords	bcrypt (cost 12) password hashes; gateway limit of 100 requests per minute per IP; email verification before sign-in	**No account lockout**: failures are not counted per account	A07, ADR-17 #1
T2	Spoofing	Forged or stolen tokens	Signed HS256 tokens, short-lived access token; refresh tokens stored as SHA-256 hashes, rotated and revoked on logout	One `SECRET_KEY` is shared by all services, so a leak from any one forges any role. A fallback key exists in the code (staging was checked and uses its own)	A02, ADR-17 #3
T3	Spoofing	Fake payment confirmation (forged PayHere webhook)	Webhook signature (HMAC) checked before any status change; invalid signature returns 400; duplicate webhooks are idempotent	Signature compared with `==`, not a constant-time comparison	A08, tests TC-FR-08
T4	Tampering	A student edits another user's data or calls admin functions	Role checks on 132 of 174 endpoints; the 42 public ones were reviewed; 403 and 401 asserted in tests; `/internal` routes blocked at the gateway	No automated test fails on a newly added unprotected route	A01
T5	Tampering	Injection through request fields	SQLAlchemy ORM; the only raw SQL is constant strings; Pydantic validation	No injection-payload test	A03

#	STRIDE	Threat	Existing controls	Residual risk / gap	Ref
T6	Tampering	Exam cheating: answers read from the start response, or one answer submitted twice	Questions are sent without the correct answers; a session is started once; tab-switch violations are recorded	The anti-cheating measure is detection, not lockdown	FR-013
T7	Tampering	Malicious uploads (receipts, materials, avatars)	MIME type checked against the file's real content; allow-list of types; asset deletion limited to own assets, with path traversal refused	Files are scanned by type only, not for malware	A08
T8	Repudiation	A user or admin denies an action (sign-in, grade change, payment approval)	Payment and enrolment records are kept; the cleanup path revokes rather than deletes enrolments	**Authentication events are not logged**; no audit log of admin actions	A09, ADR-17 #2
T9	Information disclosure	Personal data read by another user or leaked in logs	Profile reads limited to the owner or an admin; responses use per-service schemas	`notification_service` logs recipient email addresses; no central log control	A09
T10	Information disclosure	Traffic intercepted or content framed	HTTPS with a Let's Encrypt certificate; HSTS, `nosniff`, `X-Frame-Options`, `Referrer-Policy`; `Server` header removed	No Content-Security-Policy yet	A05
T11	Information disclosure	Secrets exposed in the repository or pipeline	`.env` files git-ignored; secrets only in GitHub environment secrets and Kubernetes secrets; images built from source in CI; the RabbitMQ default account removed	Secrets for the services are shared as one GitHub secret per service (an operator with the secret can read it)	A02, DDP-#73
T12	Information disclosure	Paid video or notes reached without paying	Lesson streaming checks enrolment (403 when not enrolled); videos are private YouTube embeds built from an id; Cloudinary links are signed	An enrolled student can share a video id or signed link while it is valid	FR-003
T13	Denial of service	Request flood	Rate limit at the gateway; limit per client IP	Students behind one IP (a school) share the limit; one replica per service on a small node handles about 20 concurrent clients per service before time-outs (`performance-report.md`)	NFR-009
T14	Denial of service	One service stops and takes features down	Health, startup, readiness and liveness probes restart a stuck pod; two replicas of the course service; events are queued in durable RabbitMQ queues	Other services still have a single replica; the database is a single shared Neon instance	DDP-#73
T15	Elevation of privilege	A registered user obtains the admin role	Registration forces the `student` role even if `admin` is sent (tested); role changes need an admin	Anyone who obtains the signing key (T2) can claim any role	A01, A02

#	STRIDE	Threat	Existing controls	Residual risk / gap	Ref
T16	Elevation of privilege	Compromised dependency or pipeline	`pip-audit` and `audit-ci` fail the build on new advisories; accepted advisories are documented (ADR-14); branch protection and required reviews; images tagged with the commit SHA; OIDC sign-in to Azure instead of stored cloud keys	22 accepted advisories in the frontend; no Python lock file, so builds are not fully reproducible	A06, A08
T17	Elevation of privilege	A compromised pod reaches other services or the broker	RabbitMQ and Redis are cluster-internal; the broker uses a dedicated account	**No Kubernetes network policies** (none exist in the namespace): any pod can reach every other pod, Redis and the broker	A05

4. Priorities

The risks that matter most for this release, with the planned response:

1. **T2 and T15: shared signing key.** Keep each environment's key distinct and secret (checked on staging; production to be checked); later, give services their own keys or move to asymmetric tokens.
2. **T1: no account lockout** and **T8: no authentication-event log.** Needs application code, so it is deferred with ADR-17 and listed under "Limitations & future work".
3. **T17: no network policies.** Configuration only; a default-deny policy with allowed paths from the gateway would limit the damage of a compromised pod. Not done in this release.
4. **T13 and T14: capacity.** Documented in the performance report; more replicas and a larger node are the main lever.
5. **T10: Content-Security-Policy.** Needs a report-only trial first.

5. Out of scope

Physical security of Azure data centres, the security of the third-party services themselves, and the end users' own devices and passwords.